

Приложение к ЛР-4

Машины опорных векторов и алгоритм Pegasos: интуиция, выводы и численные примеры

Пахомов В.В., группа Р3324

Аннотация

Это приложение к лабораторной работе разбирает математику SVM и алгоритма Pegasos с нуля. Мы пройдем от геометрической идеи разделяющей гиперплоскости до конкретной формулы обновления весов в коде, проследим, откуда берётся каждый множитель, и посчитаем 2 итерации Pegasos руками на маленьком примере. Цель документа — сделать так, чтобы формулы из основного отчёта перестали быть страшными и стали логичным следствием простой геометрической идеи.

Содержание

1	Зачем нужен SVM и какую задачу он решает	1
2	Геометрический смысл: максимизация зазора (margin)	2
2.1	Что такое зазор	2
2.2	Задача максимизации зазора	3
3	Soft-margin и hinge loss	3
3.1	Что делать, если данные не разделимы	3
3.2	От slack-переменных к hinge loss	4
4	Pegasos: стохастический субградиентный спуск	5
4.1	Что такое субградиент	5
4.2	Полный градиент по объёму n	6
4.3	Алгоритм Pegasos шаг за шагом	6
4.4	Откуда взялся множитель $(1 - \eta_t \lambda)$	6
4.5	Почему $\eta_t = 1/(\lambda t)$	7
5	Численный пример: 2 итерации Pegasos руками	7
6	Связь $\{0, 1\} \rightarrow \{-1, +1\}$	9
7	Метрики качества и ROC-кривая	9
7.1	Матрица ошибок и базовые метрики	9
7.2	Macro vs micro усреднение	10
7.3	ROC-кривая и AUC	10
7.4	Sigmoid для перевода в вероятности	11
8	Что важно запомнить	11

1 Зачем нужен SVM и какую задачу он решает

У нас есть набор объектов $x_1, \dots, x_n \in \mathbb{R}^d$ (в нашем случае $d = 90$ — средние амплитуды по 18 каналам и 5 временным окнам) и метки $y_i \in \{-1, +1\}$ (Target / NonTarget). Нужно найти функцию $f: \mathbb{R}^d \rightarrow \{-1, +1\}$, которая по новому объекту скажет его класс.

Линейный классификатор это самый простой выбор:

$$f(x) = \text{sign}(w \cdot x + b),$$

где $w \in \mathbb{R}^d$ — вектор весов, $b \in \mathbb{R}$ — смещение, $w \cdot x = \sum_j w_j x_j$ — обычное скалярное произведение.

Геометрически уравнение $w \cdot x + b = 0$ задаёт *гиперплоскость* (в 2D это прямая, в 3D — плоскость, в d -мерном — $(d-1)$ -мерная плоскость). Точки, у которых $w \cdot x + b > 0$, лежат по одну сторону — мы говорим $f(x) = +1$. У которых < 0 — по другую, $f(x) = -1$.

Интуиция

Представьте 2D-плоскость с двумя облаками точек: красные слева, синие справа. Между ними можно провести бесконечно много прямых, разделяющих классы. SVM ищет ту, которая лежит **ровно посередине** и максимально далеко от обоих облаков. Это даёт ей лучшую способность обобщать: малое возмущение объекта не пересечёт границу.

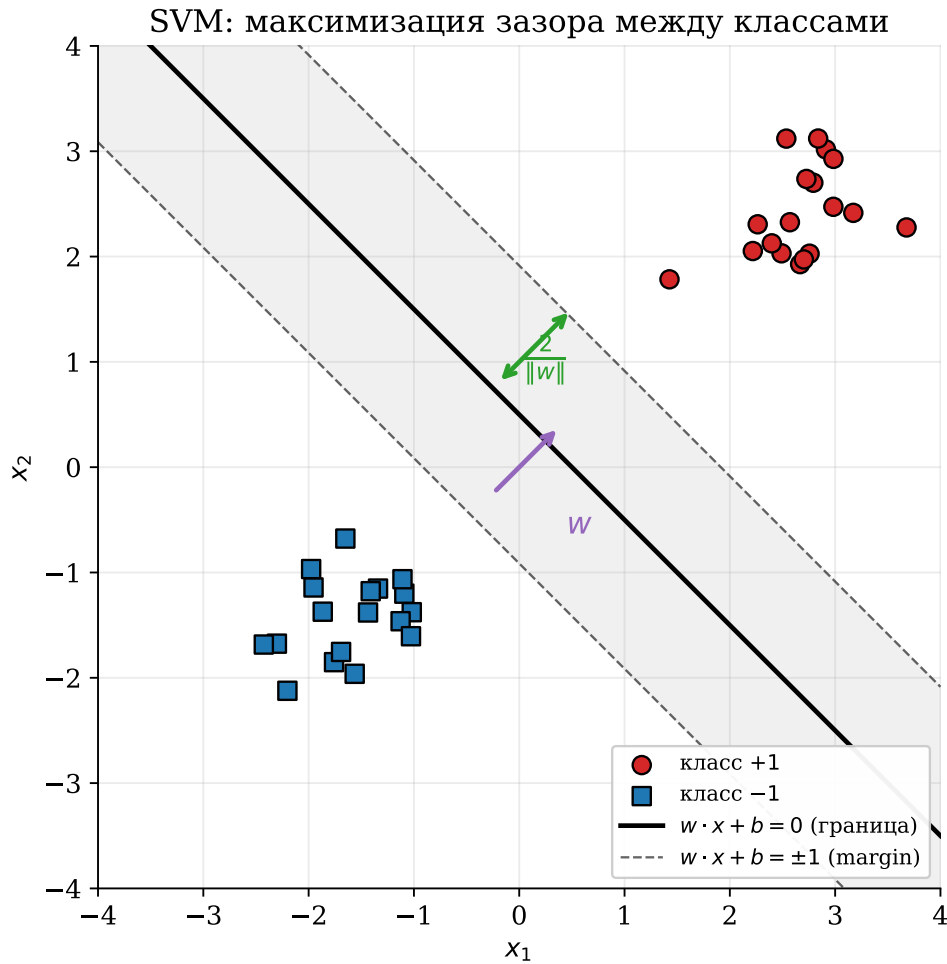


Рис. 1: Геометрический смысл SVM: чёрная линия — разделяющая гиперплоскость $w \cdot x + b = 0$, пунктирные — параллельные ей плоскости $w \cdot x + b = \pm 1$. Зазор между ними равен $2/\|w\|$, и SVM максимизирует это расстояние. Вектор w перпендикулярен границе.

2 Геометрический смысл: максимизация зазора (margin)

2.1 Что такое зазор

Возьмём гиперплоскость $H = \{x \in \mathbb{R}^d : w \cdot x + b = 0\}$. Расстояние от произвольной точки x_i до этой гиперплоскости считается по формуле:

$$d(x_i, H) = \frac{|w \cdot x_i + b|}{\|w\|},$$

где $\|w\| = \sqrt{w_1^2 + \dots + w_d^2}$ — евклидова норма.

Вывод

Откуда берётся $\|w\|$ в знаменателе. Вектор w перпендикулярен гиперплоскости (это базовое свойство уравнения $w \cdot x + b = 0$). Если из точки x_i опустить перпендикуляр на H , то он пойдёт вдоль $w/\|w\|$ — единичного направления. Длина проекции вектора $x_i - x_0$ (где x_0 — любая точка на H) на это направление:

$$\frac{(x_i - x_0) \cdot w}{\|w\|} = \frac{w \cdot x_i - w \cdot x_0}{\|w\|} = \frac{w \cdot x_i + b}{\|w\|},$$

так как $w \cdot x_0 = -b$ для любой точки на гиперплоскости. Модуль берётся, чтобы расстояние было неотрицательным.

Зазор (margin) объекта x_i с меткой $y_i \in \{-1, +1\}$ это знаковое расстояние:

$$\gamma_i = \frac{y_i(w \cdot x_i + b)}{\|w\|}.$$

Если $\gamma_i > 0$, точка классифицирована верно (метка совпадает со стороной). Если $\gamma_i < 0$ — ошибка. Величина $|\gamma_i|$ — это насколько *уверенно* классификатор отвечает: чем больше, тем дальше точка от границы.

2.2 Задача максимизации зазора

SVM хочет максимизировать минимальный по выборке зазор:

$$\max_{w,b} \min_{i=1,\dots,n} \frac{y_i(w \cdot x_i + b)}{\|w\|}.$$

Эта формулировка имеет неприятную особенность: если умножить w и b на любое положительное число $c > 0$, гиперплоскость не изменится, но числитель и знаменатель умножатся на c , и значение зазора останется тем же. То есть оптимум не единственен.

Стандартный фикс — зафиксировать масштаб условием $\min_i y_i(w \cdot x_i + b) = 1$. Это значит, что ближайшие к границе точки имеют ненормированный зазор ровно 1, а минимальный нормированный зазор тогда $1/\|w\|$. Максимизировать $1/\|w\|$ — то же, что минимизировать $\|w\|^2/2$.

Получаем классическую формулировку SVM (hard-margin):

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{при условиях} \quad y_i(w \cdot x_i + b) \geq 1 \quad \forall i.$$

Интуиция

$\|w\|^2$ маленькое $\Leftrightarrow 1/\|w\|$ большое $\Leftrightarrow$ зазор большой. Поэтому минимизация $\|w\|^2$ — это **и есть** максимизация зазора при фиксированном условии $y_i(w \cdot x_i + b) \geq 1$. Иногда новички думают, что $\|w\|^2$ — какая-то непонятная «регуляризация для борьбы с переобучением». На самом деле для SVM это не дополнительная штука, а ядро самой геометрической постановки.

3 Soft-margin и hinge loss

3.1 Что делать, если данные не разделимы

Hard-margin SVM требует, чтобы *все* объекты лежали правильно с зазором ≥ 1 . На реальных данных (особенно EEG, где шум большой) это невозможно. Идея soft-margin: разрешить нарушения, но штрафовать за них.

Введём **слабые переменные** $\xi_i \geq 0$, которые показывают, насколько объект i нарушает условие зазора:

$$y_i(w \cdot x_i + b) \geq 1 - \xi_i.$$

Если $\xi_i = 0$, объект классифицирован верно с правильным зазором. Если $0 < \xi_i < 1$ — классифицирован верно, но внутри margin-полосы. Если $\xi_i \geq 1$ — классифицирован неверно. Целевая функция теперь штрафует ξ_i :

$$\min_{w,b,\xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i, \quad \xi_i \geq 0, \quad y_i(w \cdot x_i + b) \geq 1 - \xi_i.$$

$C > 0$ — параметр, балансирующий: **большое** C = жёстко штрафует ошибки, узкий margin, риск переобучения; **малое** C = мягко, широкий margin, может недообучить.

3.2 От slack-переменных к hinge loss

Минимальное $\xi_i \geq 0$, удовлетворяющее $\xi_i \geq 1 - y_i(w \cdot x_i + b)$, это:

$$\xi_i^* = \max(0, 1 - y_i(w \cdot x_i + b)).$$

Подставив в целевую функцию, получаем **безусловную задачу**:

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i(w \cdot x_i + b))$$

Функция $\max(0, 1 - m)$, где $m = y_i(w \cdot x_i + b)$ — это и есть **hinge loss** (loss «петлёр»):

$$\ell_{\text{hinge}}(m) = \max(0, 1 - m) = \begin{cases} 0, & m \geq 1 \quad (\text{зазор} \geq 1, \text{ штраф нулевой}) \\ 1 - m, & m < 1 \quad (\text{штраф растёт линейно}) \end{cases}$$

Интуиция

График hinge loss:

- На $m \geq 1$ ровный нуль: объект далеко за границей с правильной стороны, всё хорошо.
- На $0 \leq m < 1$: правильно классифицирован, но внутри margin-полосы. Штраф растёт.
- На $m < 0$: ошибка, штраф > 1 , растёт линейно.

В точке $m = 1$ есть «излом», производная не существует. Это важно для алгоритма — понадобится *субградиент* вместо обычного градиента.

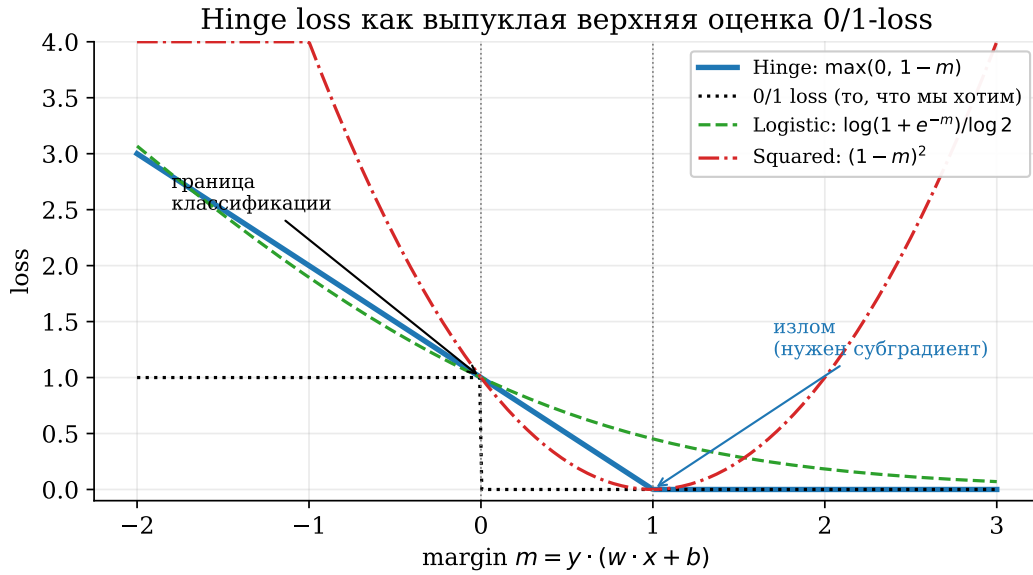


Рис. 2: Hinge loss как функция margin'a $m = y(w \cdot x + b)$. Чёрный пунктир — «правильный» 0/1 loss, который мы хотим минимизировать, но не можем (он недифференцируем и не выпуклый). Hinge — его выпуклая верхняя оценка. На $m \geq 1$ штраф ноль (объект далеко за margin), при $m < 1$ растёт линейно. В точке $m = 1$ излом — именно поэтому в Pegasos используется субградиент. Для сравнения показаны logistic loss (что использует логрег) и квадратичный loss.

В отчёте используется нормировка на n и параметр $\lambda = 1/C$ вместо C . Это эквивалентно с точностью до масштаба:

$$L(w, b) = \frac{\lambda}{2} \|w\|^2 + \frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i(w \cdot x_i + b)).$$

При $C = 1$ имеем $\lambda = 1$, то есть оба слагаемых в нашей реализации идут с равным весом.

4 Pegasos: стохастический субградиентный спуск

4.1 Что такое субградиент

Hinge $\ell(m) = \max(0, 1 - m)$ не дифференцируем в точке $m = 1$. Слева производная -1 , справа 0 . **Субградиент** — это любое значение между ними (включая концы):

$$\partial \ell(m) = \begin{cases} \{-1\}, & m < 1 \\ [-1, 0], & m = 1 \\ \{0\}, & m > 1 \end{cases}$$

В алгоритме мы выбираем удобный элемент субградиента. На границе $m = 1$ берём 0 — это даёт более «гладкое» поведение. То есть на практике:

$$\frac{\partial \ell_i}{\partial w} = \begin{cases} -y_i x_i, & y_i(w \cdot x_i + b) < 1 \quad (\text{объект внутри margin}) \\ 0, & \text{иначе} \end{cases}$$

$$\frac{\partial \ell_i}{\partial b} = \begin{cases} -y_i, & y_i(w \cdot x_i + b) < 1 \\ 0, & \text{иначе} \end{cases}$$

Вывод

Вычислим градиент $\ell_i = \max(0, 1 - y_i(w \cdot x_i + b))$ по w на участке $m < 1$ (то есть $1 - m > 0$, hinge активен):

$$\frac{\partial}{\partial w} (1 - y_i(w \cdot x_i + b)) = -y_i \frac{\partial(w \cdot x_i)}{\partial w} = -y_i x_i.$$

По b аналогично: $\partial/\partial b (1 - y_i(w \cdot x_i + b)) = -y_i$.

Регуляризация $\frac{\lambda}{2} \|w\|^2$ даёт $\partial/\partial w = \lambda w$ (по b зависимости нет).

4.2 Полный градиент по объёму n

Если бы мы хотели делать честный градиентный спуск, надо было бы суммировать по всем n объектам:

$$\nabla_w L = \lambda w - \frac{1}{n} \sum_{i:m_i < 1} y_i x_i.$$

Для $n = 1728$ это нормально, но для миллиона объектов уже тяжело. Pegasos предлагает **стохастическую** версию: на каждой итерации берём *один случайный* объект, считаем градиент по нему и шагаем.

4.3 Алгоритм Pegasos шаг за шагом

Инициализация: $w_1 = 0$, $b_1 = 0$.

На итерации $t = 1, 2, \dots, T$:

1. Выбрать случайный индекс $i \in \{1, \dots, n\}$.
2. Вычислить шаг обучения $\eta_t = \frac{1}{\lambda t}$ (убывает по гиперболе).
3. Вычислить margin $m_i = y_i(w_t \cdot x_i + b_t)$.
4. Если $m_i < 1$ (hinge активен):

$$w_{t+1} = (1 - \eta_t \lambda) w_t + \eta_t y_i x_i, \quad b_{t+1} = b_t + \eta_t y_i.$$

5. Иначе (объект далеко за margin, hinge = 0):

$$w_{t+1} = (1 - \eta_t \lambda) w_t, \quad b_{t+1} = b_t.$$

4.4 Откуда взялся множитель $(1 - \eta_t \lambda)$

Стандартный шаг градиентного спуска для одной точки:

$$w_{t+1} = w_t - \eta_t \nabla_w \ell_i(w_t).$$

Здесь $\nabla_w \ell_i$ — градиент *полного* loss по одному объекту, включая регуляризацию:

$$\nabla_w \ell_i = \lambda w_t + \begin{cases} -y_i x_i, & m_i < 1 \\ 0, & m_i \geq 1 \end{cases}$$

Подставляем:

$$w_{t+1} = w_t - \eta_t (\lambda w_t - y_i x_i \cdot [m_i < 1]) = (1 - \eta_t \lambda) w_t + \eta_t y_i x_i \cdot [m_i < 1].$$

Множитель $(1 - \eta_t \lambda)$ — это **сжатие весов**, происходящее каждую итерацию от регуляризации λ . Это работает как «забывание»: каждый шаг текущий w слегка сжимается к нулю, тем самым $\|w\|$ остаётся ограниченной. Прибавление $\eta_t y_i x_i$ происходит только когда объект внутри margin — это «коррекция в сторону этого объекта».

4.5 Почему $\eta_t = 1/(\lambda t)$

Это не магия — это **оптимальная** скорость для сильно выпуклых функций. Целевая функция:

$$L(w) = \frac{\lambda}{2} \|w\|^2 + \frac{1}{n} \sum \ell_i(w)$$

сильно выпукла с параметром λ (из-за квадратичного члена $\lambda/2 \cdot \|w\|^2$). Теория стохастического градиентного спуска говорит, что для λ -сильно выпуклых функций оптимальный шаг $\eta_t = c/(\lambda t)$ даёт скорость сходимости $\mathbb{E}[L(w_T)] - L(w^*) = O(\log T/T)$.

Интуиция

Шаг должен убывать, иначе w будет «прыгать» вокруг оптимума и не сойдётся. Но не слишком быстро — иначе остановимся, не дойдя до оптимума.

$1/t$ — золотая середина: суммарный путь $\sum_t 1/t$ расходится (медленно, как $\log T$), значит мы можем добраться куда угодно, но скорость убывает, поэтому колебания вокруг оптимума затухают.

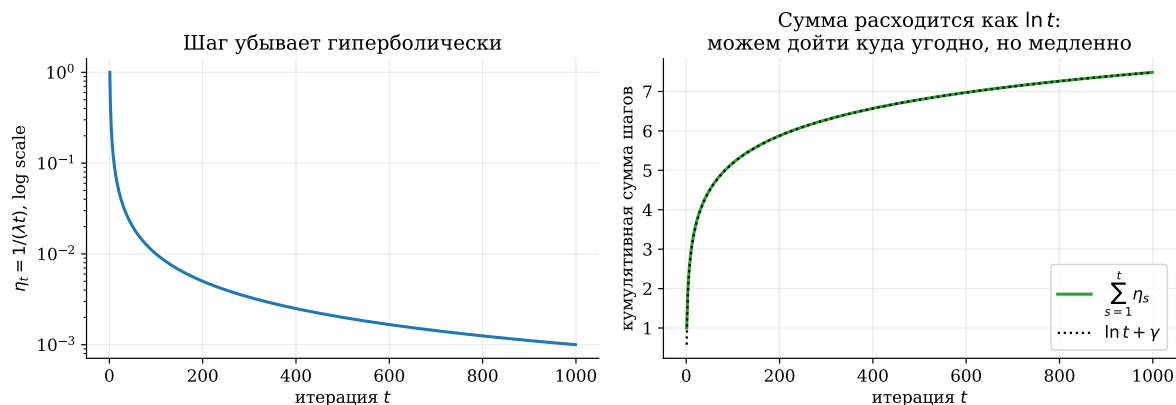


Рис. 3: Слева: $\eta_t = 1/(\lambda t)$ убывает гиперболически (log-шкала по y). Справа: кумулятивная сумма $\sum_{s=1}^t \eta_s$ растёт как $\ln t$ — расходящийся, но очень медленно. Это идеальный баланс: бесконечное число шагов даёт неограниченный пробег, но колебания вокруг оптимума затухают.

5 Численный пример: 2 итерации Pegasos руками

Возьмём игрушечную задачу в 2D. Четыре точки, два класса:

i	x_i	y_i	описание
1	(2, 1)	+1	Target справа сверху
2	(3, 2)	+1	Target справа сверху
3	(-1, -1)	-1	NonTarget слева снизу
4	(-2, 0)	-1	NonTarget слева снизу

Параметры: $\lambda = 1$, $w_1 = (0, 0)$, $b_1 = 0$.

Итерация $t = 1$. $\eta_1 = 1/(\lambda \cdot 1) = 1$.

Случайно выбрали $i = 1$, объект $x_1 = (2, 1)$, $y_1 = +1$.

Margin: $m_1 = y_1(w_1 \cdot x_1 + b_1) = 1 \cdot (0 \cdot 2 + 0 \cdot 1 + 0) = 0$. Это < 1 , hinge активен.

Обновление:

$$w_2 = (1 - 1 \cdot 1) \cdot (0, 0) + 1 \cdot 1 \cdot (2, 1) = (0, 0) + (2, 1) = (2, 1).$$

$$b_2 = 0 + 1 \cdot 1 = 1.$$

Итерация $t = 2$. $\eta_2 = 1/(1 \cdot 2) = 0,5$.

Случайно выбрали $i = 3$, объект $x_3 = (-1, -1)$, $y_3 = -1$.

Margin: $m_3 = -1 \cdot (2 \cdot (-1) + 1 \cdot (-1) + 1) = -1 \cdot (-2 - 1 + 1) = -1 \cdot (-2) = 2$. Это > 1 — объект уже хорошо классифицирован, hinge неактивен.

Обновление (только сжатие):

$$w_3 = (1 - 0,5 \cdot 1) \cdot (2, 1) = 0,5 \cdot (2, 1) = (1, 0,5).$$

$$b_3 = 1.$$

Численный пример

После двух итераций граница: $1 \cdot x_1 + 0,5 \cdot x_2 + 1 = 0$, или $x_2 = -2x_1 - 2$. Эта прямая отделяет $\{x_1, x_2\}$ от $\{x_3, x_4\}$ верно, и зазор уже есть. Каждая следующая итерация будет уточнять её.

Этот процесс на практике выглядит как на рисунке ниже: уже после 100 итераций граница становится хорошо центрированной, а к 1500 итерациям margin прижимается к ближайшим точкам каждого класса:

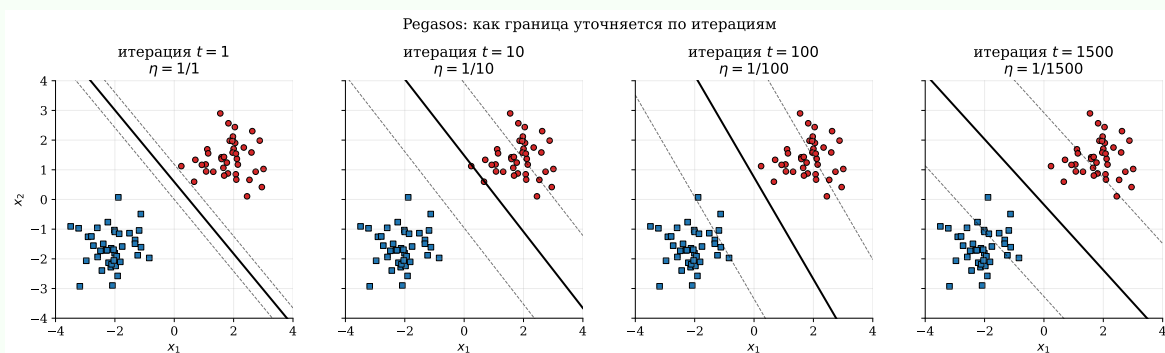


Рис. 4: Pegasos на двух классах. Сплошная линия — граница $w \cdot x + b = 0$, пунктир — параллельные ± 1 -плоскости (margin). На итерации 1 граница случайна (мы видели один объект), на 10 уже грубо в правильную сторону, на 100 близка к финальной, на 1500 margin прижимается к ближайшим точкам.

Заметьте: на шаге 2 **веса уменьшились** (с $(2, 1)$ до $(1, 0,5)$), хотя мы ничего не «выучили нового». Это работает регуляризация. Если бы мы не встречали больше объектов внутри margin, w постепенно сжимался бы к нулю — именно это и есть смысл регуляризации.

6 Связь $\{0, 1\} \rightarrow \{-1, +1\}$

В коде есть строка:

```
y_train_svm = 2 * y_train - 1
```

Это переводит метки. Откуда: если $y \in \{0, 1\}$, то $2y - 1 \in \{-1, +1\}$. Зачем это нужно?

Hinge loss $\max(0, 1 - y \cdot (w \cdot x + b))$ имеет смысл только для $y \in \{-1, +1\}$. Если бы мы оставили $y \in \{0, 1\}$, то для отрицательного класса $y = 0$ выражение $y \cdot (w \cdot x + b) = 0$, и $\text{hinge} = \max(0, 1 - 0) = 1$ *всегда* (независимо от предсказания), что бессмысленно.

При $y \in \{-1, +1\}$ всё работает: для положительного класса хотим $w \cdot x + b > 0$ ($\text{margin} > 0$), для отрицательного $w \cdot x + b < 0$, и $y \cdot (w \cdot x + b) > 0$ в обоих случаях, когда ответ верный.

7 Метрики качества и ROC-кривая

7.1 Матрица ошибок и базовые метрики

После предсказания $\hat{y}$ на тесте формируется матрица 2×2 :

		Предсказано	
		+	-
Истина	+	TP	FN
	-	FP	TN

- **TP** (true positive) — истинно положительный: $y = 1, \hat{y} = 1$.
- **TN** (true negative) — истинно отрицательный.
- **FP** (false positive, ошибка I рода) — $y = 0, \hat{y} = 1$.
- **FN** (false negative, ошибка II рода) — $y = 1, \hat{y} = 0$.

Метрики:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (\text{доля верных, обманчива при дисбалансе})$$

$$\text{Precision} = \frac{TP}{TP + FP} \quad (\text{точность: доля верных среди предсказанных +})$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (\text{полнота: доля найденных среди истинных +})$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (\text{гармоническое среднее})$$

Интуиция

Зачем precision и recall, если есть accuracy? В нашей задаче P300 истинных Target всего 1 из 6 эпох. Тривиальный классификатор «всегда NonTarget» даёт accuracy $\approx 0,83$ — звучит хорошо, но мы вообще не нашли ни одного Target! Поэтому:

- **Recall** говорит, сколько Target мы поймали из всех настоящих. Низкий recall = пропустили P300-вспышки.
- **Precision** говорит, насколько мы доверяем нашему «Target»-предсказанию. Низкий precision = много ложных тревог.

- **F1** штрафует за провал в любой из двух — хорошее единое число для дисбалансных задач.

7.2 Macro vs micro усреднение

Для бинарной задачи macro считает precision/recall для каждого класса отдельно и усредняет с равным весом. Это правильно при дисбалансе: важно качество *обоих* классов одинаково.

7.3 ROC-кривая и AUC

SVM возвращает не только метку, но и *уверенность* — значение $w \cdot x + b$ (decision function). Чем больше, тем сильнее модель уверена в положительном классе. Можем варьировать порог τ и считать предсказанием $\hat{y} = +1 \iff w \cdot x + b > \tau$.

При разных τ получаются разные TPR и FPR:

$$\text{TPR} = \frac{TP}{TP + FN} = \text{Recall}, \quad \text{FPR} = \frac{FP}{FP + TN}.$$

- При $\tau = -\infty$ всё предсказано $+$: $TP = \text{все Target}$, $FP = \text{все NonTarget}$. $\text{TPR} = 1$, $\text{FPR} = 1$.
- При $\tau = +\infty$ всё $-$: $TP = 0$, $FP = 0$. $\text{TPR} = 0$, $\text{FPR} = 0$.

ROC-кривая — параметрическая кривая $(\text{FPR}(\tau), \text{TPR}(\tau))$ при пробегании τ от $+\infty$ до $-\infty$. Она идёт из $(0, 0)$ в $(1, 1)$.

AUC (Area Under Curve) — площадь под этой кривой:

- $\text{AUC} = 1$ — идеальный классификатор (есть такой порог, при котором $\text{TPR} = 1$, $\text{FPR} = 0$).
- $\text{AUC} = 0,5$ — случайный (кривая = диагональ $y = x$).
- $\text{AUC} < 0,5$ — хуже случайного (можно инвертировать метки).

Вероятностная интерпретация AUC. Возьмём случайный объект x^+ из класса $+$ и случайный x^- из класса $-$. AUC — это вероятность, что классификатор поставит x^+ выше:

$$\text{AUC} = \Pr(f(x^+) > f(x^-)).$$

То есть $\text{AUC} = 0,995$ в нашей задаче значит: в 99,5% случаев случайный Target ранжируется выше случайного NonTarget.

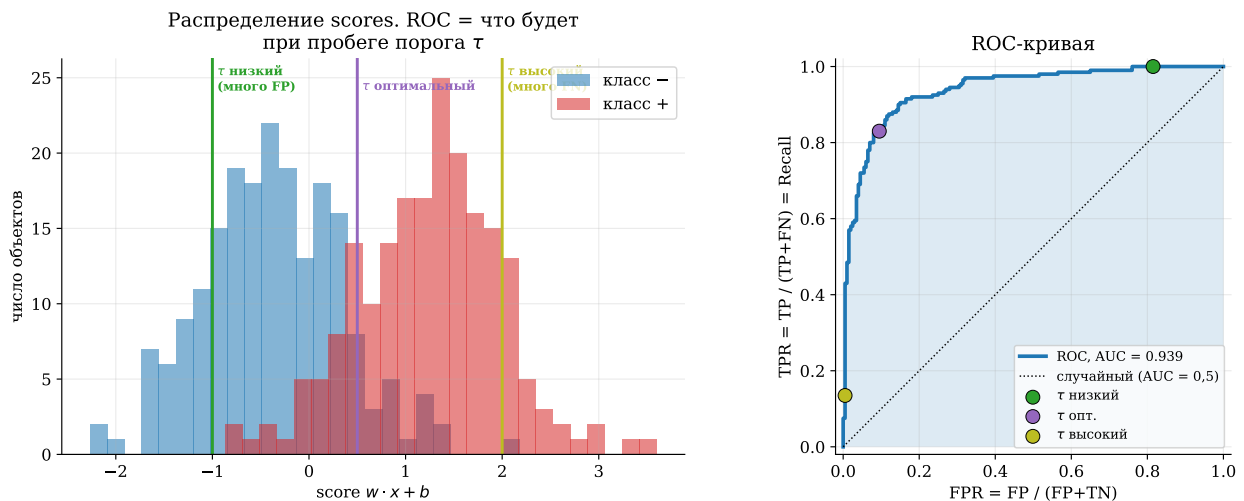


Рис. 5: Что такое ROC-кривая. Слева: распределения scores $w \cdot x + b$ для двух классов. Три порога τ показывают разные trade-off: низкий ловит почти всех Target ценой ложных тревог, высокий — мало ложных, но много пропусков. Справа: соответствующая ROC. Каждая точка кривой — результат одного порога; AUC — площадь под всей кривой и не зависит от выбора τ .

7.4 Sigmoid для перевода в вероятности

В коде используется

$$\hat{p}(x) = \frac{1}{1 + e^{-(w \cdot x + b)}}.$$

Это *искусственная* калибровка — SVM не возвращает вероятности по построению, в отличие от логистической регрессии. Для ROC-кривой подходит *любая* монотонная функция от $w \cdot x + b$, так что сигмоида не меняет ROC (поэтому AUC одинаков с просто decision function).

8 Что важно запомнить

- SVM ищет гиперплоскость, максимизирующую **зазор** (margin), то есть расстояние от границы до ближайших точек.
- Минимизация $\|w\|^2$ — это *не дополнительная* регуляризация, а сама математическая постановка максимизации margin'a.
- Soft-margin вводит slack-переменные, и они превращаются в **hinge loss** $\max(0, 1 - m)$.
- Hinge не дифференцируем в одной точке, поэтому Pegasos использует **субградиент**.
- Формула Pegasos $w \leftarrow (1 - \eta\lambda)w + \eta yx$ — это просто стохастический градиент с регуляризацией.
- $\eta_t = 1/(\lambda t)$ — оптимальная скорость для сильно выпуклых задач.
- Метки $\{0, 1\} \rightarrow \{-1, +1\}$ — из-за устройства hinge loss.
- AUC — вероятность правильного ранжирования случайной пары (x^+, x^-) , не зависит от выбора порога.